

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики

Кафедра вычислительной математики и информационных
прикладных технологий

Направление 01.04.02 Прикладная математика и информатика

Отчёт

по производственной практике (научно-исследовательская работа)

Тема	Разработка аппаратной реализации 2D графического ускорителя
Отчетный семестр	4

Обучающийся	2 курс, 11 группа, Старухин Д.М.
Руководитель от кафедры	к.ф.-м.н., доц. Медведева О.А.

Воронеж 2026

Содержание

Введение	3
Постановка задачи	4
Актуальность поставленной задачи	5
1. Обзор архитектур графических процессоров	6
1.1. Обзор архитектур графических процессоров компании Nvidia	6
1.2. Обзор архитектур графических ускорителей компании AMD	12
1.3. Заключение обзора	20
2. Архитектура ядра	23
2.1. Различие и сходство архитектур ядер CPU и GPU	23
2.2. Интерфейсы графического ядра	24
2.2.1. Интерфейс передачи данных и инструкций (AHB)	25
2.2.2. Интерфейс передачи данных с регистровым файлом (APB)	26
2.3. Поток	26
2.3.1. Классификация параллельных вычислительных моделей (SIMT vs SIMD, MIMD).	27
Заключение	29
Литература	30

Введение

В этой работе будет представлена аппаратная архитектура реализации ядра для графических ускорителей. Также, в этой работе будут рассмотрены другие архитектуры графического ядра у ведущих компаний в разработке графических ускорителей. С учётом уже существующих архитектур графических ядер, в работе будет представлена собственная архитектура графического ядра.

Постановка задачи

Компанией АО «ПКК» Миландр была поставлена задача разработать аппаратную реализацию 2D графического ускорителя. Перед началом аппаратной разработки необходимо описать архитектуру, которой должен соответствовать сложно-функциональный (СФ) блок.

При разработке архитектуры учитываются проделанные ранее обзор и анализ 2D графических ускорителей [1]. Разработка ведется с учётом предложенной ранее [2] архитектурой графического ускорителя.

Перед тем как разработать аппаратную реализацию 2D графического ускорителя, необходимо и важно разобраться в предназначении и структуре различных компонентов IP-блока. Одним из таких компонентов, является графическое ядро.

Графическое ядро имеет довольно сложную архитектуру, которое состоит из элементов, которые должны корректно соответствовать друг другу. Поэтому, перед разработкой графического ядра, нужно разработать его архитектуру, с учётом разработок графических ядер других кампаний.

Актуальность поставленной задачи

В современном мире, в условиях развития нейросетей и импортозамещения, создание собственных специализированных графических и вычислительных архитектур является довольно важной задачей.

Нейросети все больше вовлекаются во многие аспекты, начиная от развлекательной индустрии, заканчивая научными исследованиями. Искусственный интеллект образовал огромный спрос видеокарт, с помощью которых можно собрать вычислительный сервер, на которых, можно развернуть нейросеть для решения различных задач. Видеокарта состоит из большого количества элементов, одним из которых является графический процессор [3]. графический процессор обладает большим количеством ядер, которые выполняют огромный объем вычислений с плавающей точкой.

Помимо искусственного интеллекта, графические ядра также могут использоваться в вычислениях рендеринга графики, обработки видео или фото, а также в различных научных вычислениях, где требуется рассчитывать тысячи параллельных математических вычислений.

Разработка архитектуры графического ядра позволит снизить зависимость от зарубежных архитектур, таких, как AMD, ARM Mali и Nvidia. Благодаря разработке IP-блока, можно достичь создания доверенных систем, которые не будут содержать в себе наличия аппаратных «закладок» и недокументированных функций в критически важной инфраструктуре.

1. Обзор архитектур графических процессоров

Перед тем, как заниматься разработкой архитектуры графического ядра, стоит обратить внимание на уже существующие разработки различных компаний.

1.1. Обзор архитектур графических процессоров компании

Nvidia

Одна из компаний, занимающаяся разработкой графических ускорителей является Nvidia, которая с 1993 года занимается разработкой аппаратных ускорителей вычисления [4]. Nvidia за более 30 лет разработала большое количество микроархитектур для графических процессоров.

Одной из первых архитектур, разработанных Nvidia являлась архитектура Rankie, которая использовалась с 2003 по 2005 года в графических процессорах. Одним из графических процессоров, разработанным на микроархитектуре Rankie являлся NVIDIA NV30. Он обладал 125 миллионами транзисторами с техпроцессом 130нм. Графический процессор на тот момент обладала 4 пиксельными шейдерами, 3 вершинных шейдера, 8 блоков наложения текстур и 4 блоками вывода рендеринга (ROP (Render Output Unit)) [5].

Блок диаграмма архитектуры графического процессора NVIDIA NV30 представлена на рис. 1:

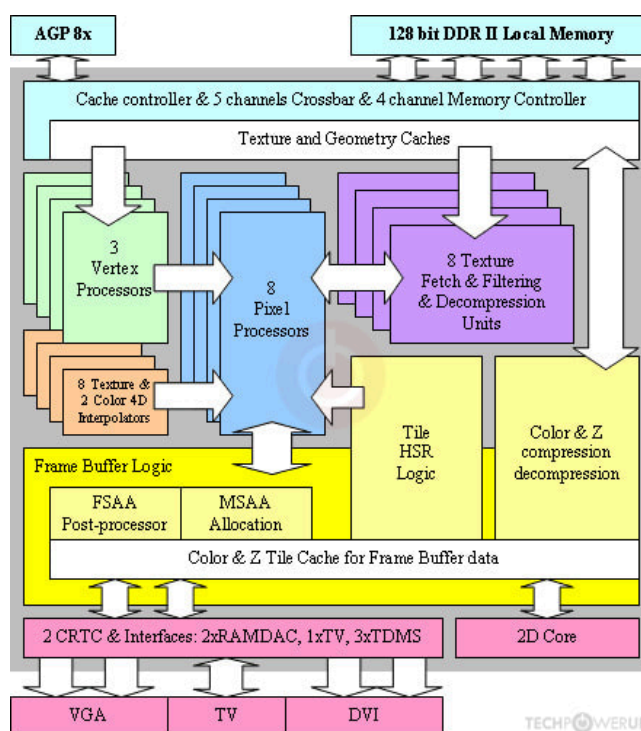


Рис. 1 — Блок схема архитектуры графического процессора NVIDIA NV30

По рис. 1 заметно, что архитектура очень похожа на современные 2D-графические ускорители, разрабатываемые в микроэлектронике. Понятие графического ядра тогда не существовало, вместо графического ядра все задачи выполняли вершинные и пиксельные процессоры. До 2006 года видеокарты использовали жестко разделенные процессоры, которые выполняли специализированные задачи. Но на сегодняшний день, вершинным и пиксельным процессорам пришли на замену унифицированные шейдерные ядра. Такие ядра выдавали более высокую производительность чем шейдеры, в последствии чего, будущие архитектуры графических процессоров стали основываться на использовании большого количества универсальных блоков. Такие блоки стали называться потоковыми процессорами (SP) [6].

Первая архитектура, которая стала использовать SP, как основу в графических процессорах стала архитектура Tesla, представленная с 2006 года. Данная архитектура в первую очередь отличалась от архитектуры Rarke использованием SP блоков, что являлось в новинку в архитектурах графических процессоров компании Nvidia. Архитектура графического процессора изображена на рис. 2:

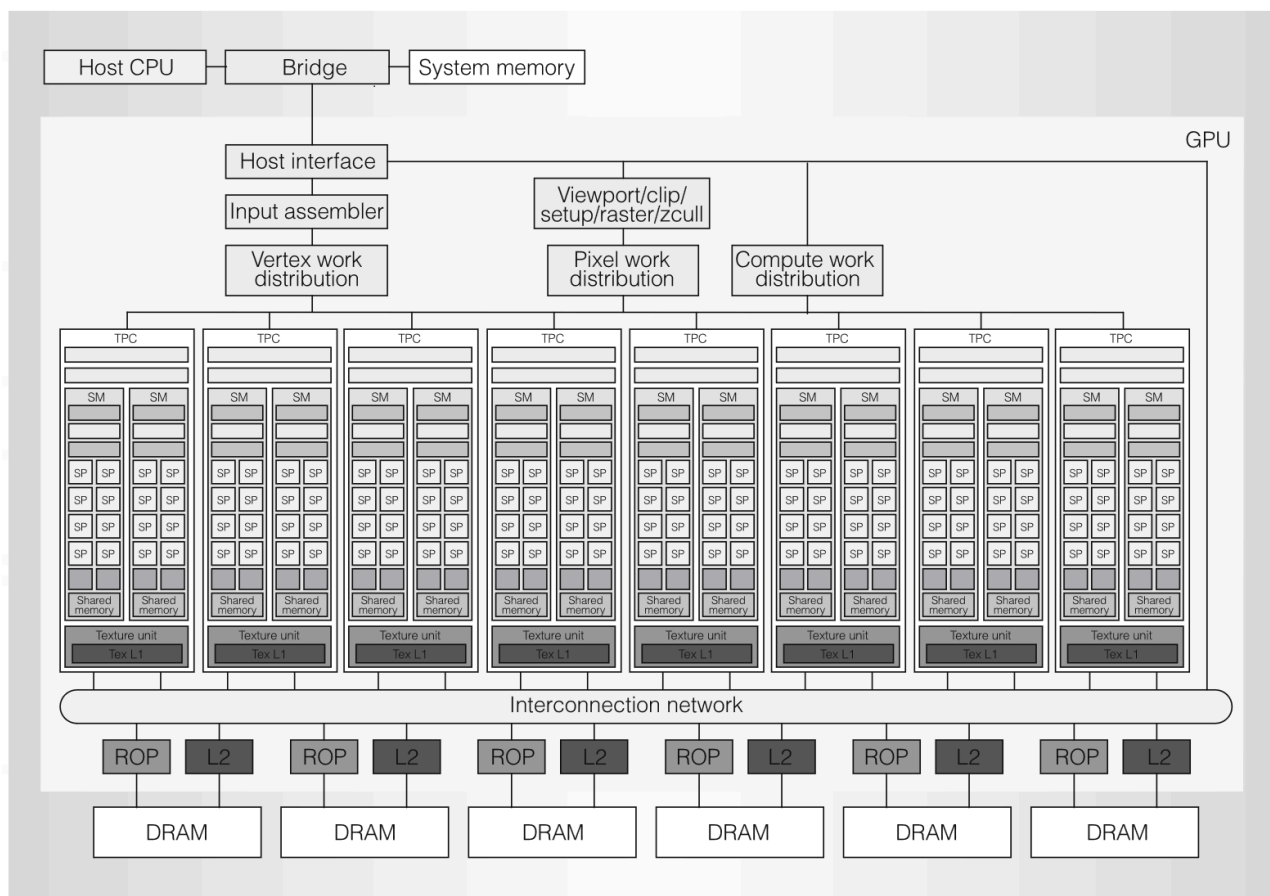


Рис. 2 — Архитектура графического процессора Tesla

Можно заметить, что архитектура Tesla сильно различается от Ranki. Теперь всю работу выполняют кластеры обработки текстур (TPC - texture/processor cluster). В графическом процессоре находится множество TPC и каждый из них является комплексным компонентом в архитектуре. Компонент TPC, в свою очередь, состоит из геометрического контроллера (Geometric controller), текстурного блока (Texture unit) контроллера потокового мультимикропроцессора (SMC) и самих потоковых мультимикропроцессоров (SM). Каждый потоковый мультимикропроцессор представляет собой набор из кешей (I cache и C cache), обработчика прерываний (MT issue), Блок специальных функций (SFU), общая память и потокового процессора (SP) [7]. Архитектура TPC представлена на рис. 3:

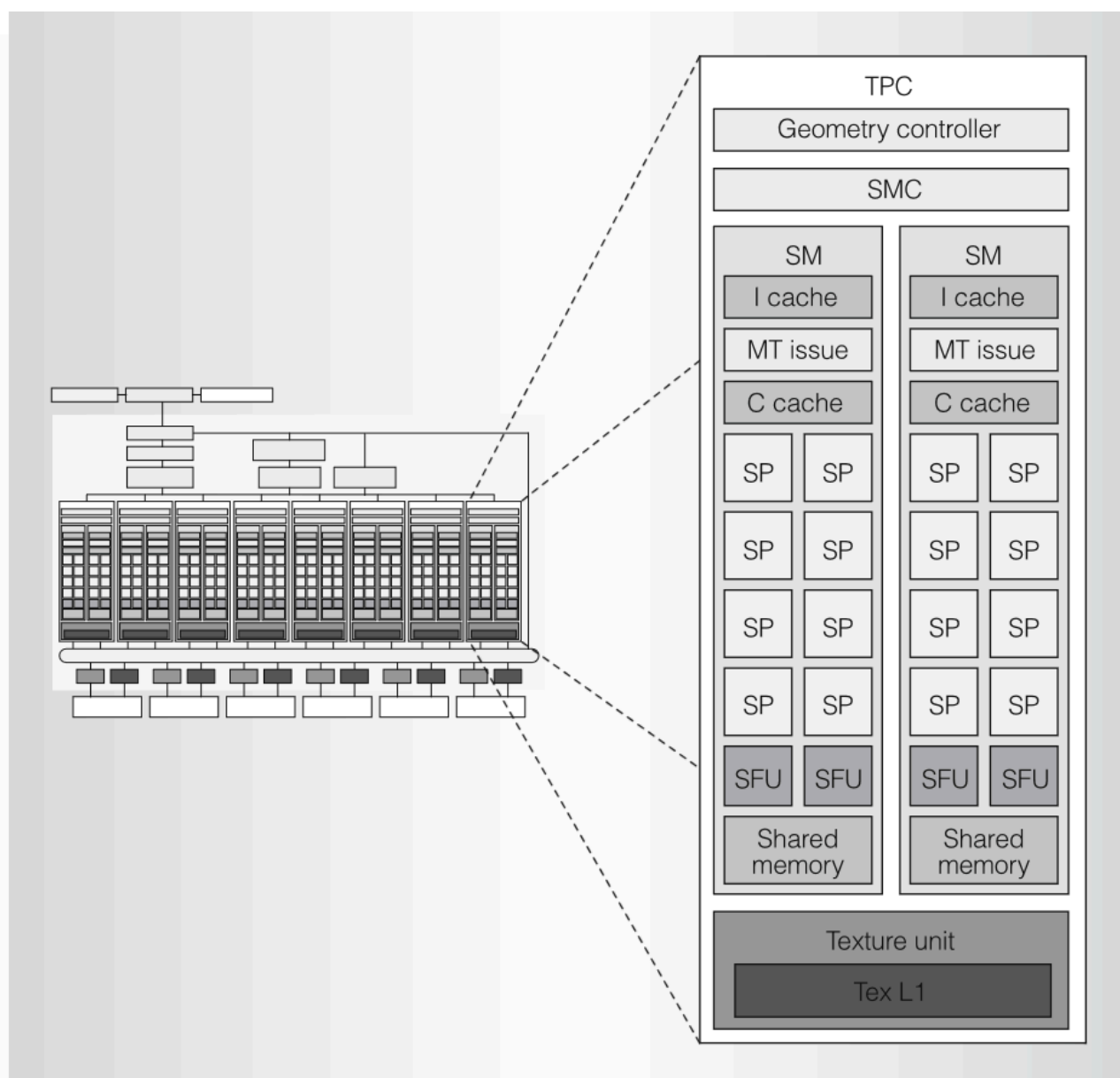


Рис. 3 — Архитектура графического процессора Tesla

По своей сути, основную работу с вычислениями выполняли SP, все сложные функции и операции, такие как вычисления синуса, косинуса или экспоненты отдавались на вычисление блоку SFU. В следующих архитектурах компании Nvidia, SP заменяются на CUDA-ядра, которые будут являться архитектурой параллельных вычислений и основой для разработки графических процессоров. Архитектура, в которой появились CUDA-ядра является архитектурой Fermi, используемая в видеокартах, начиная с 2010 года. Одной из главных особенностей стало появление CUDA-ядер. На рис. 4 можно увидеть компонент SM в архитектуре Fermi:

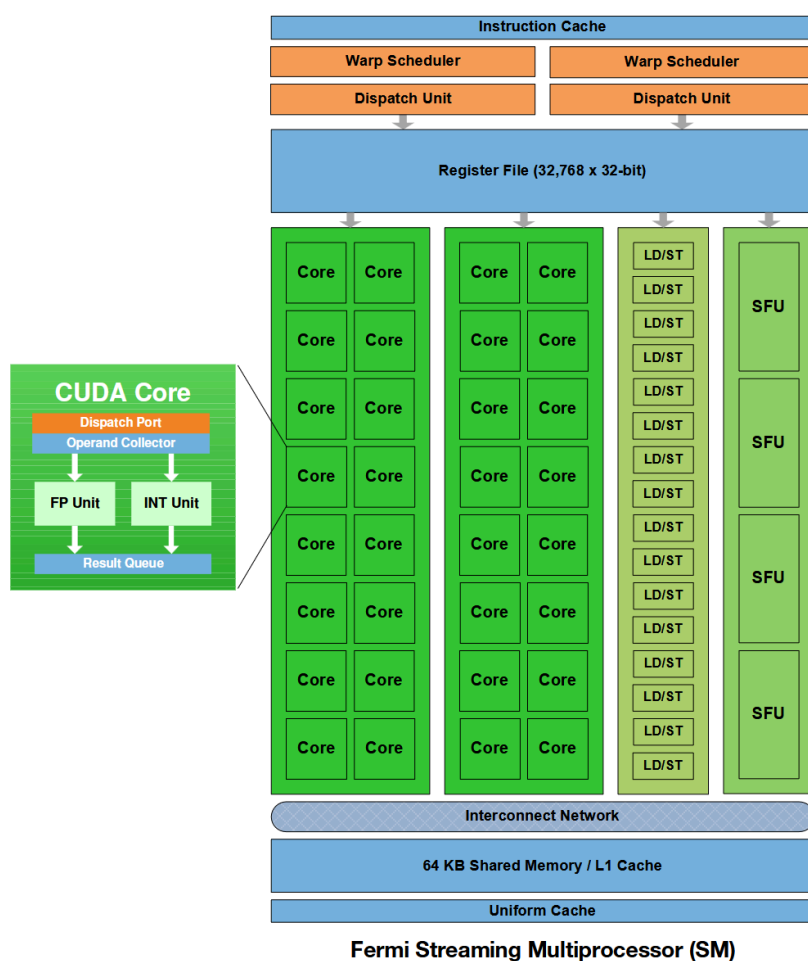


Рис. 4 — Поточковый мультипроцессор архитектуры Fermi (SM Fermi)

В потоковом мультипроцессоре на место SP появились CUDA-ядра. CUDA-ядро состоит из порта для блока распределения и выдачи задач, операнда-коллектора, вычислительных элементов и очереди для результатов выполнения поставленных задач ядру. Вычислительные элементы являются FP-вычислительные блоки (FPU), для вычисления чисел с плавающей точкой и INT-вычислительные блоки (ALU), для вычислений целочисленных чисел. Также в потоке мультипроцессора появился блок LD/ST (Load/Store), который является блоком загрузки и сохранения. Также в SM появился реги-

стовый файл (Register File), для сохранения промежуточных значений, планировщик (Warp Scheduler) и блок выдачи и распределения задач (Dispatch Unit) [8].

Теперь рассмотрим более современную микроархитектуру - Turing, разработанную Nvidia для видеокарт, начиная с 2018 года. Данная архитектура представляла собой первую микроархитектуру, которая поддерживала трассировку лучей в реальном времени и применялась в потребительских продуктах [9]. Одним из графических процессоров, разработанных на микроархитектуре Turing являлся графический процессор NVIDIA TU102. С его архитектурой можно ознакомиться на рис. 5:

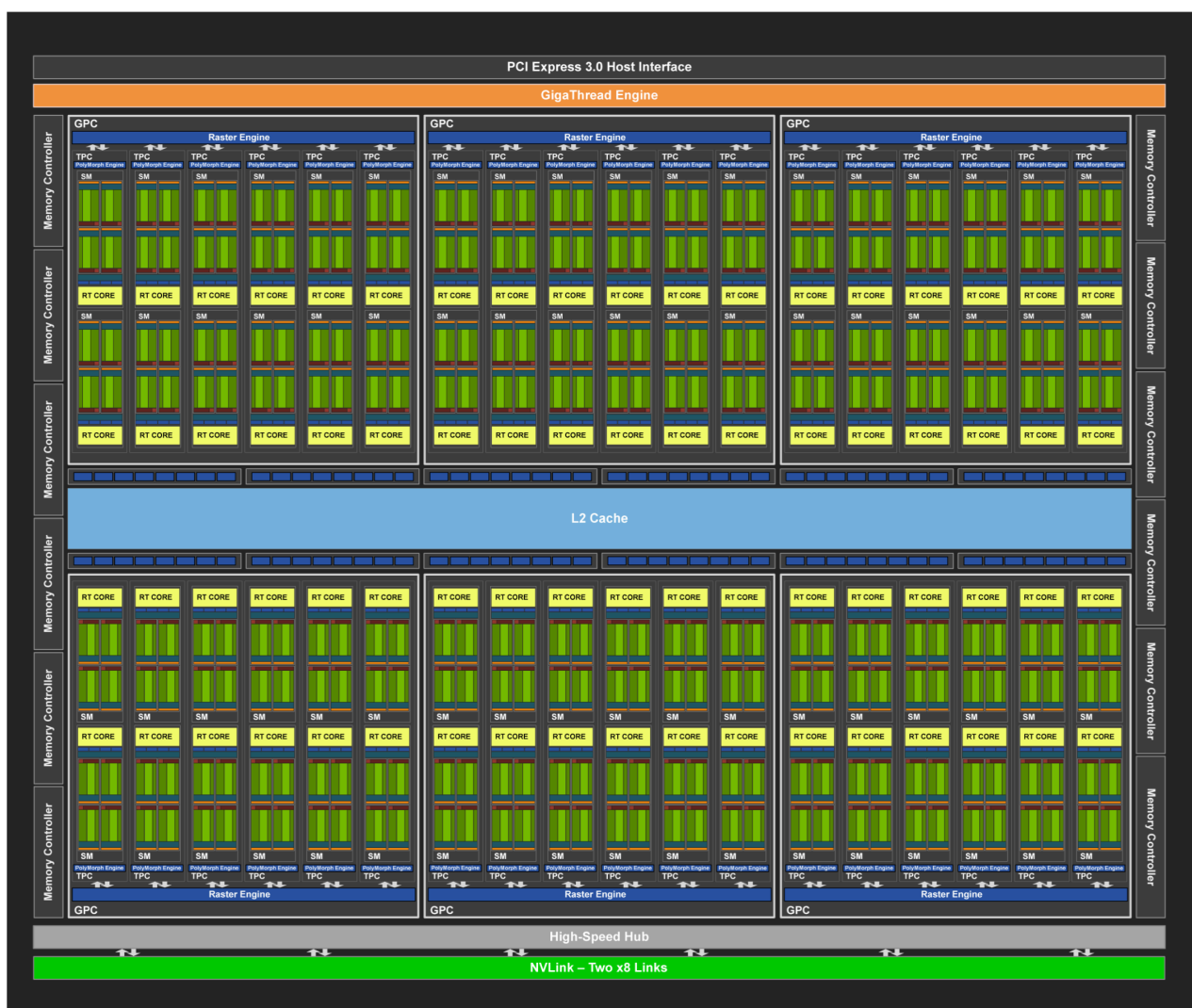


Рис. 5 — Архитектура графического процессора NVIDIA TU102

Такая архитектура является уже сильно улучшенной по сравнению с предыдущими представленными. За 8 лет, количество графических ядер у графических процессоров заметно увеличилось. На архитектуре графического процессора NVIDIA TU102 можно увидеть около 4608 универсальных ядер (CUDA-ядер), 72 ядер для выполнения рей-трейсинга и 576 тензорных ядер. Один графический процессор может

иметь 6 вычислительных графических кластеров (GPC), в каждом из которых по 12 потоковых мультипроцессоров (SM). В каждом мультипроцессоре есть 1 ядро для выполнения рей-трейсинга и 4 вычислительных единицы, в каждом из которых есть по 16 вычислительных CUDA-ядер и 2 тензорных ядра. В новых потоковых мультипроцессорах помимо CUDA-ядер появились ядра для рей-трейсинга (RT Core) и тензорные ядра (Tensor Core) для умножения и свёртки матриц [10]. Структуру потокового мультипроцессора (SM) Turing NVIDIA TU102 можно увидеть на рис. 6:

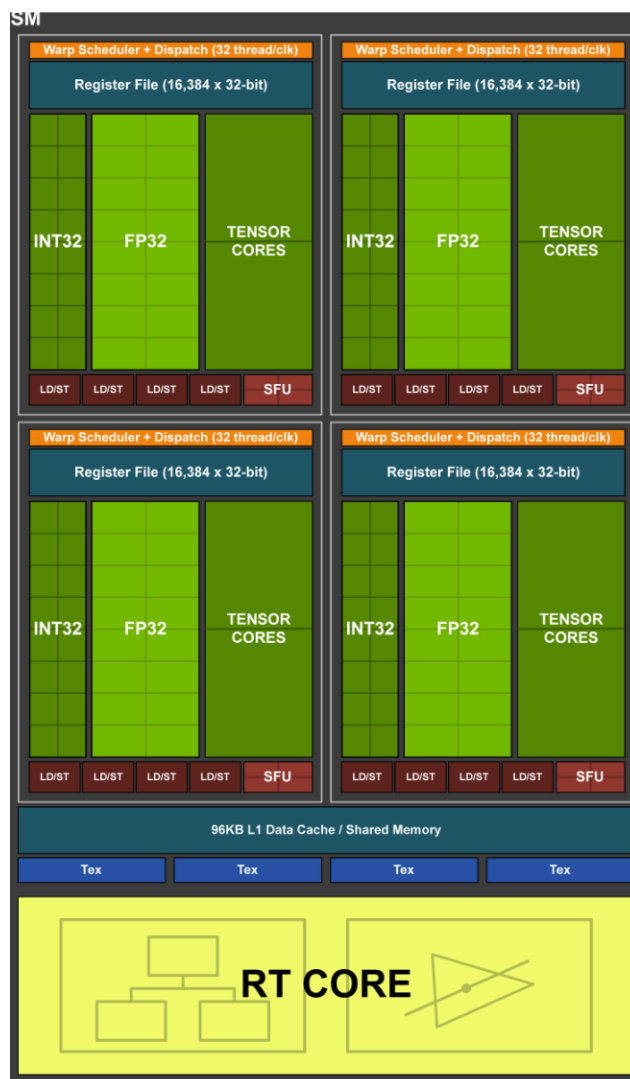


Рис. 6 — Архитектура SM NVIDIA TU102

Количество LD/ST-блоков знатно меньше, чем вычислительных элементов у потокового мультипроцессора. Если в архитектуре Fermi использовалось на 2 ядра 1 LD/ST блок, то количество ядер на 1 блок загрузки и сохранения увеличилось вдвое. В будущих архитектурах, структура потокового мультипроцессора в общем останется неизменной. В архитектуре Blackwell заметно общее увеличение SM-блоков и улучшение тензорных и рей-трейсинговых ядер.

1.2. Обзор архитектур графических ускорителей компании AMD

Компания Advanced Micro Devices, Inc. или же AMD является американским производителем интегральных микросхем и электроники. Данная компания в том числе занимается не только разработкой центральных процессоров, но и графических. Компания была основана в 1969 году, но первый графический процессор под брендом AMD стал ATI Radeon серии R100, выпущенный в 2000 году. По своей сути, графическими процессорами на тот момент занималась компания ATI, которая в будущем, в последствии была выкуплена самой AMD [11].

ATI занималась разработкой видеокарт для вычислительных машин, начиная с конца 80-ых, когда использовались фиксированные функции для вычислений графики. Не было шейдеров, вычисления происходили быстро, но при этом крайне ограничено. Видеокарты были довольно ограниченными в плане использования, ведь не допускали написания собственного кода, а лишь позволяли настраивать предустановленные этапы. AMD описывает эту эпоху, как Fixed Function (Фиксированные функции).

На смену первой эпохе пришла эпоха Simple Shaders или же эпоха простых шейдеров. Появились программируемые шейдеры. Это позволяло писать программистам различные алгоритмы и программы, расширяя возможности графического процессора [12]. Шейдеры разделялись на вершинные и пиксельные. В такой эпохе могла возникнуть проблема, когда одни шейдеры простаивали, пока шейдеры другого типа были перегруженными. Данная эпоха давала гибкость, но при этом была не так эффективной, как следующая эпоха.

Третья эпоха называется Graphics Parallel Core или же параллельные графические ядра. Главная особенность этой эпохи заключается в унифицированной архитектуре. На смену векторным и пиксельным шейдерам приходят ядра, способные выполнять большой список задач, повышая гибкость графического процессора и уменьшая простой элементов от специализации блоков.

Все три эпохи изображены на рис. 7:

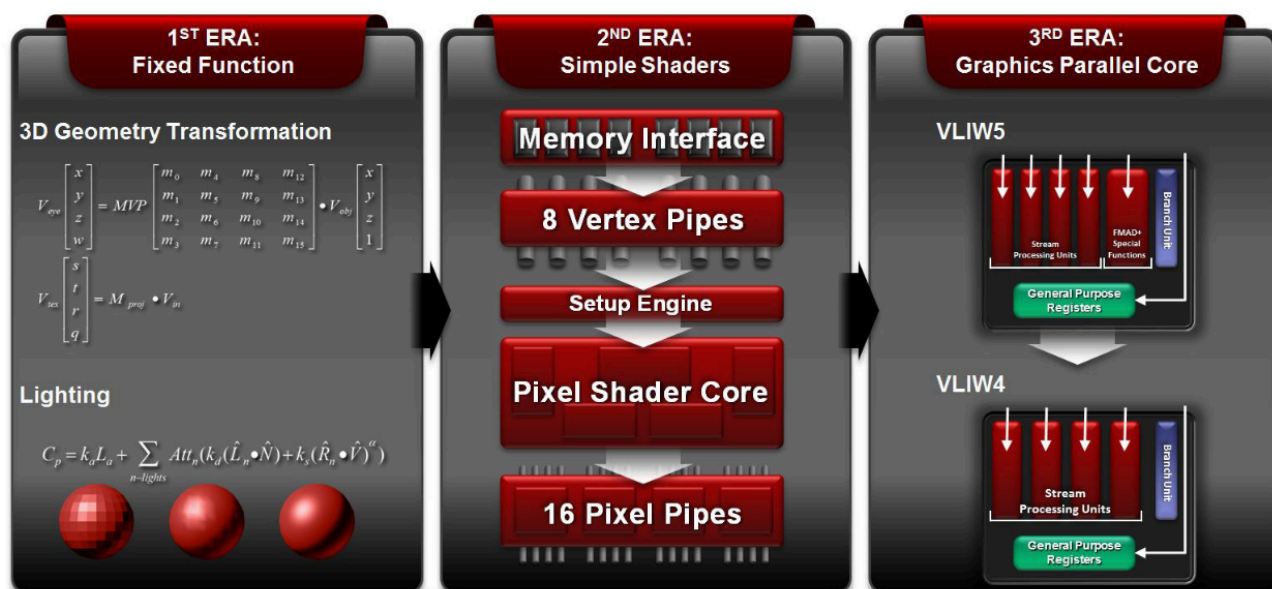


Рис. 7 — Эволюция видеокарт по классификации AMD

Рассмотрим архитектуру AMD TeraScale, которая использовалась с 2005 по 2013 год. Первый графический чип, разработанный на архитектуре TeraScale назывался R600, а все графические процессоры, основанные на R600 относились к семейству R600 (R600-Family) [13]. Архитектура семейства R600 показана на рис. 8

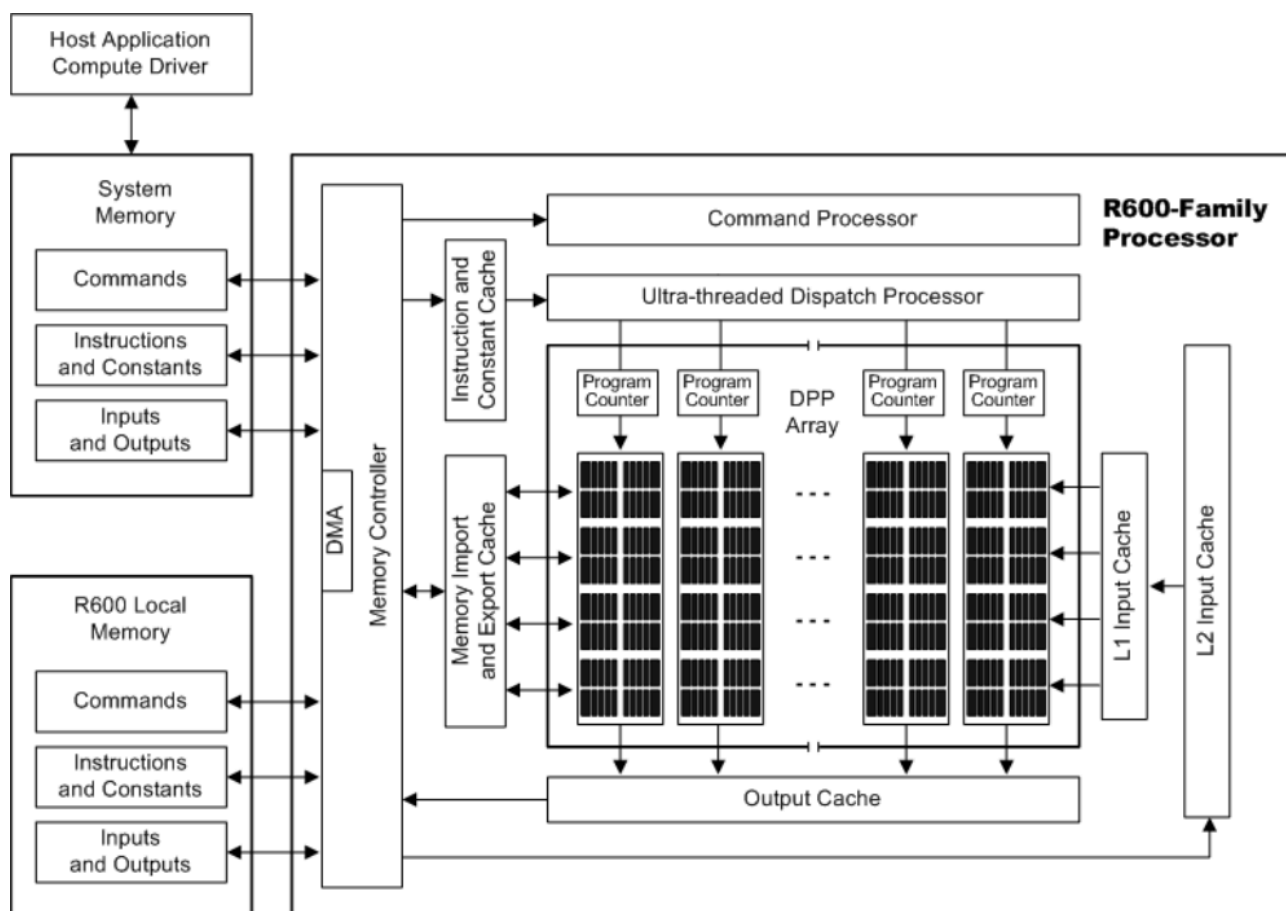


Рис. 8 — Графический процессор семейства R600

Можно увидеть, что графический процессор семейства R600 состоит из большого количества кешей, процессора команд, контроллера памяти, блока выдачи задач потокам и массива параллельных процессов (DPP - Data Parallel Processor). DPP состоит из набора SIMD-конвейеров, каждый из которых независим от других. SIMD-конвейер является набором потоков, которые выполняют вычисления над данными.

R600 семейство было первым поколением, которое применила унифицированную шейдерную архитектуру. Вместо использования отдельных вершинных и пиксельных шейдеров, графический процессор содержал в себе универсальные ядра, способные заменить специализированные шейдеры [14].

Дальше архитектуру TeraScale стали модернизировать и структура семейства процессоров Evergreen стала выглядеть иначе, чем семейство R600. Ознакомится с структурой процессора семейства Evergreen, можно увидеть на рис. 9:

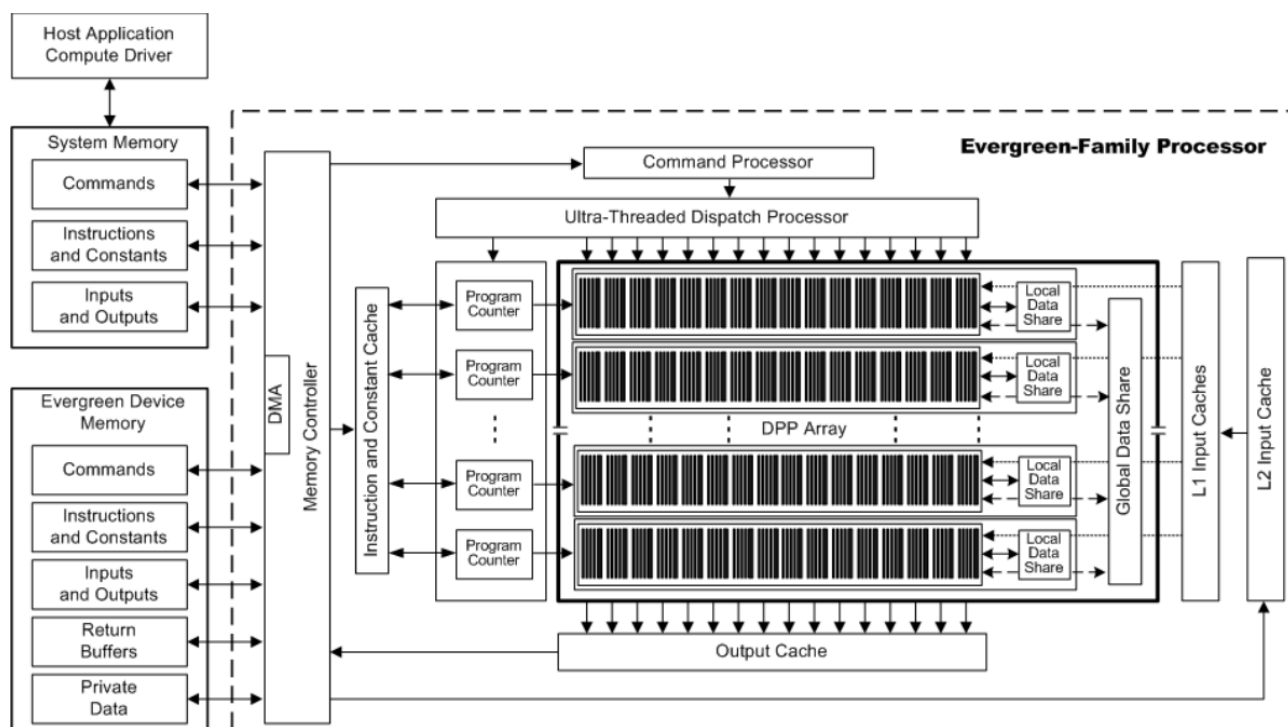


Рис. 9 — Графический процессор семейства Evergreen

Можно заметить, что у семейства графических процессоров Evergreen появились дополнительные компоненты, которые войдут в будущие архитектуры графических процессоров AMD. Одним из таких является локальная разделяемая память или Local Data Share (LDS). LDS предоставляет быструю память для обмена данными между потоками в SIMD-конвейере. При этом, также есть и память общего назначения (GDS), которая предоставляет общую память для всех вычислительных элементов в класте.

ре. Иерархию разделяемой памяти в потоковых процессорах архитектуры семейства Evergreen можно увидеть на рис. 10:

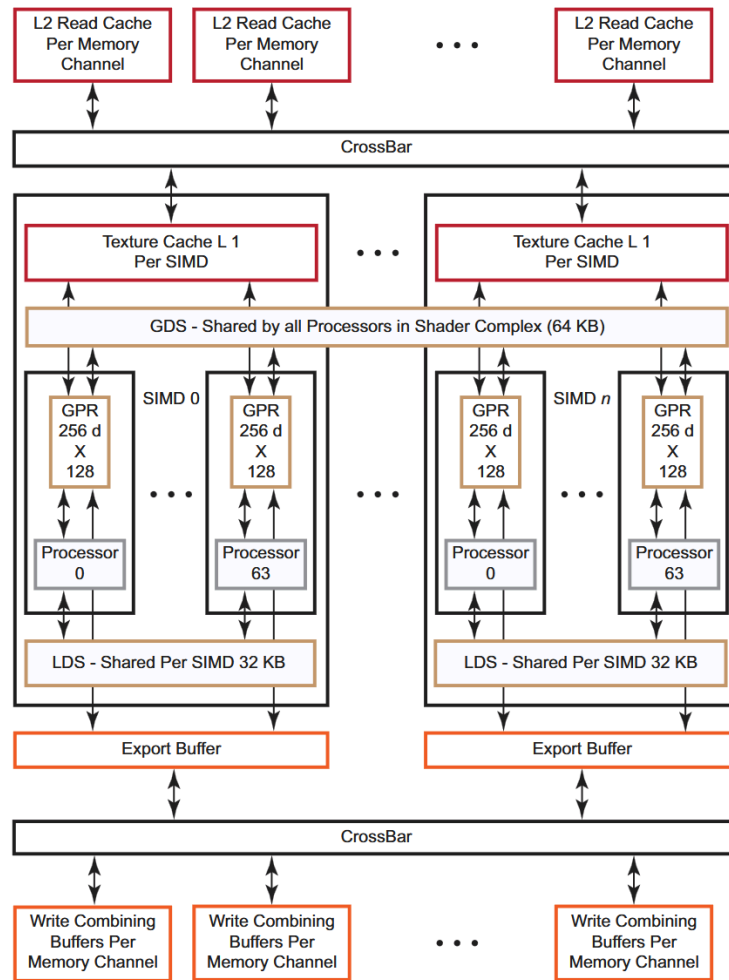


Рис. 10 — Иерархия разделяемой памяти в семействе потоковых процессоров Evergreen

Также в потоке появится сущность под названием GPR - General Purpose Registers или регистры общего назначения. Они предназначены для хранения промежуточных данных, которые используются потоками прямо во время вычислений. Главная особенность GPR заключается в сверхбыстром доступе к индивидуальным данным для каждого потока. Также благодаря GPR допускается сохранение обеспечения параллелизма в графическом ядре, так как у каждого потока есть свой блок GPR [15].

В документации архитектуры Evergreen появились следующие термины:

- Рабочий элемент (Work-item) - набор параллельно исполняемых потоков на устройстве, вызванных при

помощи команды. Рабочий элемент не зависит от другого рабочего элемента и может исполнять параллельно код с остальными. Если ему необходимо получить

данные используемые или уже обработанные любым другим рабочим элементом, он может получить их через общую память. По своей сути рабочий элемент это поток;

- Рабочая группа (Work-group) - набор взаимодействующих рабочих элементов, исполняющихся на одном

устройстве. У каждой рабочей группы есть своя рабочая память. По своей сути это и является вычислительным блоком в аппаратном виде [16]. В свою же очередь Рабочие группы организуются в волновой фронт или же в Wavefront (Warp) [17]. Количество потоков Wavefront определяют его размер. Обычно этот размер либо 32, либо 64. Все потоки, находящиеся в одном Wavefront выполняют одну и ту же инструкцию над разными потоками данных.

В архитектурах начиная с Vega, work-group имеют другую терминологию. Рабочая группа становится набором Wavefront'ом, которые имеют возможность быстро синхронизироваться между собой. Wavefront'ы также могут передавать друг другу данные через общий LDS.

После архитектуры TeraScale AMD стала выпускать графические процессоры на архитектуре GCN - Graphics Core Next, что в переводе называется «Графическое ядро следующего поколения». Данная архитектура использовалась в графических ускорителях с 2012 по 2020 год.

Одной из главных особенностей GCN - это отказ от VLIW (very long instruction word) - очень длинных машинных команд. Раньше TeraScale получала на вход команды, в которых содержалось несколько последовательных инструкций, которые нужно выполнить элементу DPP. С переходом на новую архитектуру графических процессоров, AMD решило отказаться от VLIW в пользу скалярной архитектуры набора команд (GCN Quad). Это позволило повысить производительность, за счёт более эффективного использования ресурсов, что привело и к снижению потребления. При этом, так как вычислительные компоненты графического ядра получали лишь одну инструкцию, а не VLIW, это повысило гибкость методов компиляции, которые упрощают разработку средств отладки и анализа. Для VLIW приходилось прибегать к аккуратной оптимизации, с которой можно было достичь пика производительности графического ядра. При этом, возникали конфликты с регистрами при использовании VLIW, которые необходимо было решать. GCN Quad архитектура инструкций лишала вышеперечисленных проблем и поэтому она пришла на замену VLIW [18].

Второй главной особенностью архитектуры GCN - переход из SIMD-конвейеров в вычислительные блоки (CU - Compute Unit). В одном вычислительном блоке находилось:

- sALU (Scalar ALU) и причастный к нему скалярный GPR. sALU выполняет скалярные вычисления. sALU

обрабатывает данные для всего Wavefront'а целиком, а не для отдельных потоков.

Блок выполняет вычисления индексов, ветвления и uniform операций;

- 4 SIMD-конвейера vALU (Vector ALU) и к ней причастной GPR. Выполняет векторные операции над

потоками. Каждый vALU содержит в себе 16 процессорных элементов (PE);

- Локальная память LDS;
- Доступ чтения и записи к векторной памяти кеша 1-ого уровня [19];
- Кеш инструкций и констант, которые разделены для 4-ёх CU;
- Декодировщик, захватчик команд, буфер и обработчик прерываний.

Структуру вычислительного устройства можно увидеть на рис. 11:

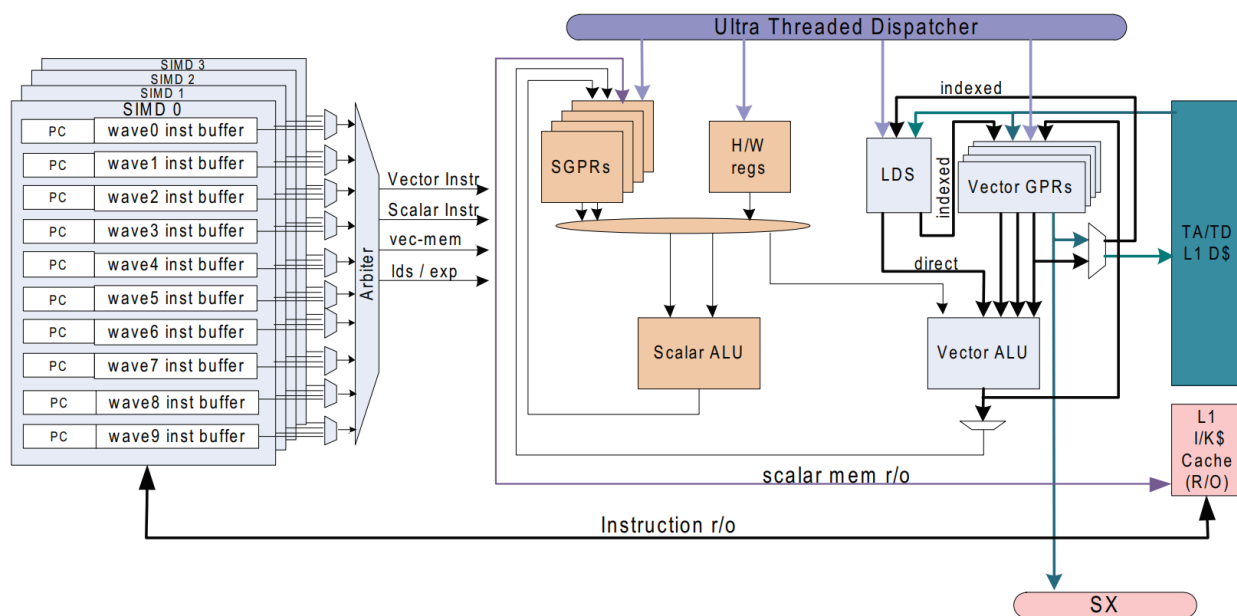


Рис. 11 — структура вычислительного блока GCN

Архитектурой GCN обладает серия графических процессоров Southern Islands (Southern Islands Series Processor, видеокарты этой серии также называют SI-GPU) Общая структура архитектуры графических процессоров серии Southern Islands показана на рис. 12:

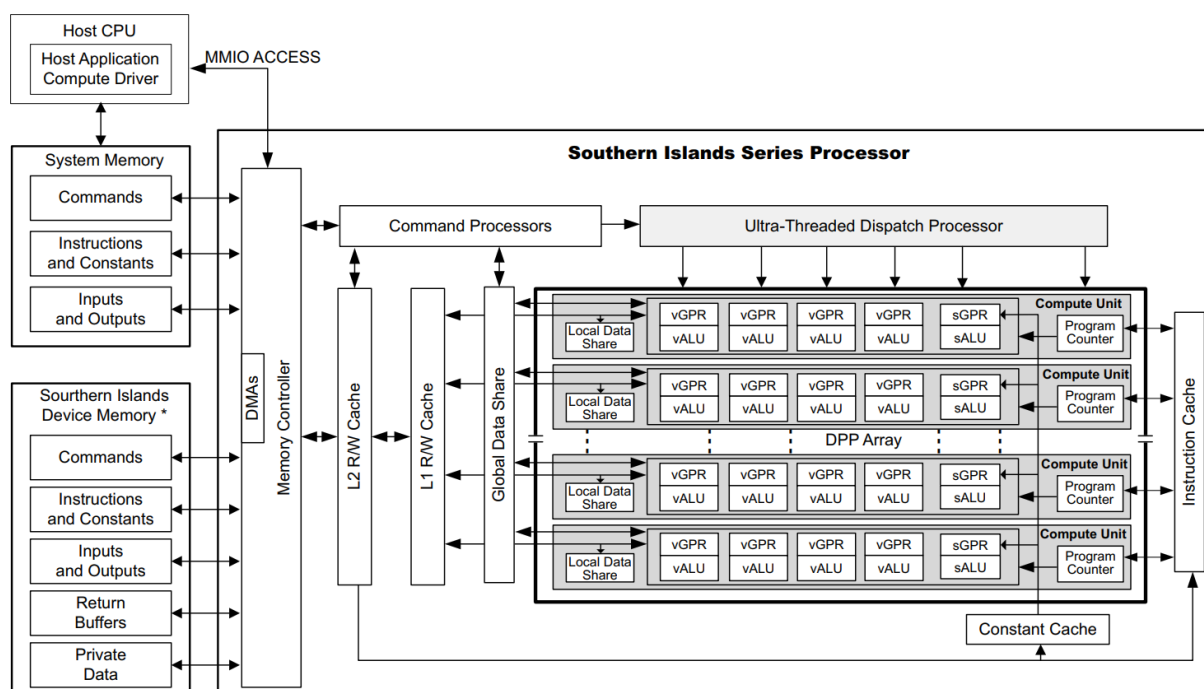


Рис. 12 — Архитектура графических ускорителей серии Southern Islands

На смену GCN, с 2019 года, приходит архитектура RDNA (Radeon DNA). RDNA обладала новой сущностью, называемой WGP (Work Group Processor) или процессор рабочей группы. WGP первой архитектуры RDNA состоит из 2 CU. В GCN поддерживался размер волнового фронта, равный 64 рабочим элементам. Архитектура RDNA поддерживает как 32-поточные, так и 64-поточные архитектуры с различными размерами Wavefront'a [20]. Количество вычислительных элементов vALU увеличено до 32, из-за чего вычисления проводились за 1 такт, вместо 4 тактов, которые наблюдались в архитектуре GCN. При этом, число вычислительных элементов на 1 CU остается неизменным.

Также иерархия кешей усложняется в архитектуре RDNA в пользу ускорения работы вычислительных блоков с памятью. С появлением WGP появляется кеш инструкций и данных 0-ого уровня, который выделен для каждого ядра. Кеш 1 уровня становится общим. Разницу архитектур GCN и RDNA можно увидеть на рис. 13:

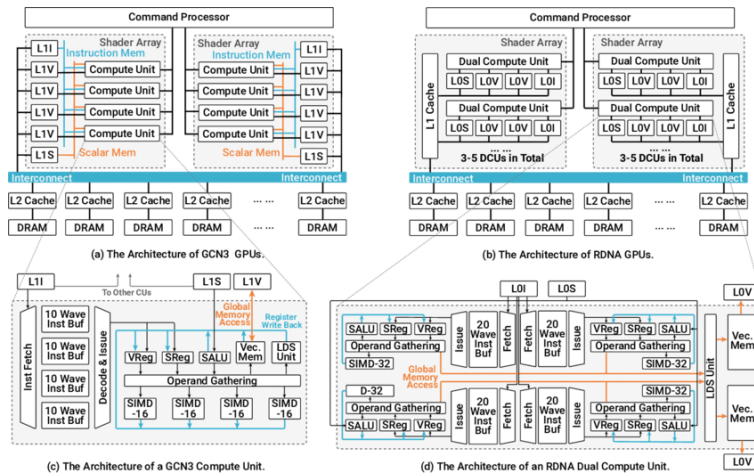


Рис. 13 — Сравнение архитектур графических ускорителей GCN и RDNA

Архитектура RDNA изображена на :

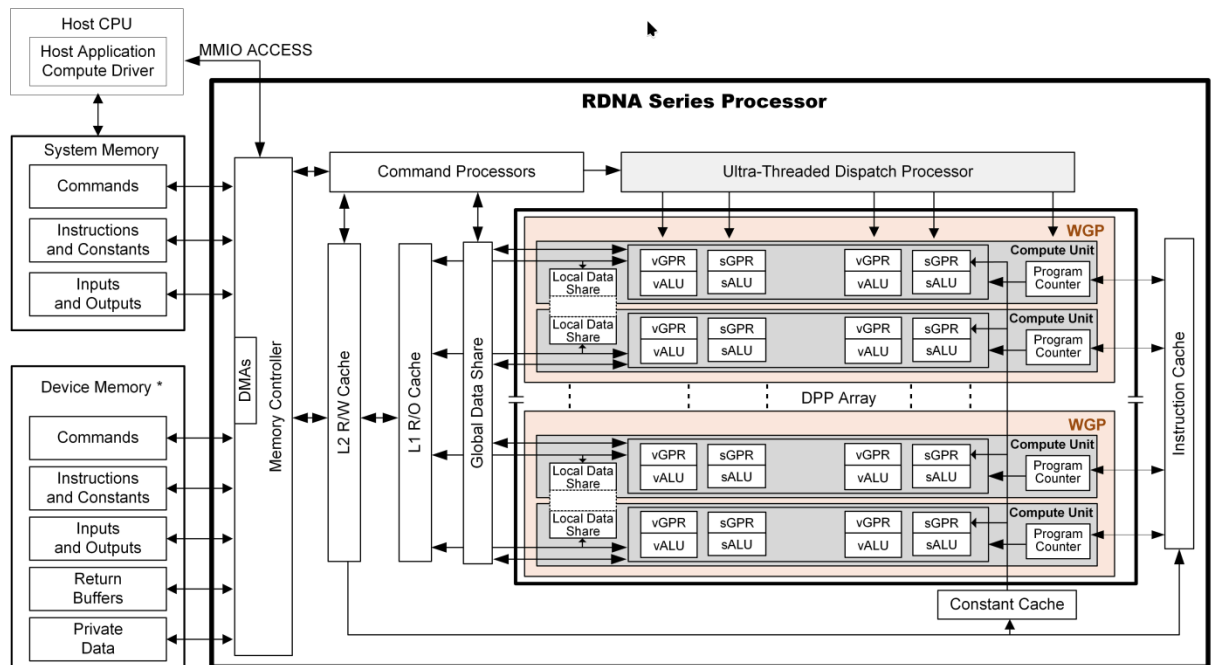


Рис. 14 — Архитектура графического процессора RDNA

В RDNA SGPR хранит в себе различные данные состояний ядра, промежуточные значения и управления потоками. Некоторые из сигналов, хранящиеся в SGPR выводятся напрямую к необходимым блокам, минуя интерфейс чтения данных. Потоки с помощью SGPR можно выключать, образуя возможность создавать шейдерами с блоками условий (создание ветвей). Также, к примеру, выводится напрямую результат работы сравнения для обеспечения скорости отработки операций и чтения результатов сравнения.

При этом архитектура RDNA обеспечивает обратную совместимость с архитектурой GCN и поддерживает 7 базовых типов инструкций:

- скалярные вычисления;

- скалярный доступ к памяти;
- векторные вычисления;
- векторный доступ к памяти;
- инструкции ветвления (бранчи);
- команды экспорта данных;
- служебные сообщения.

1.3. Заключение обзора

Благодаря рассмотрению различных архитектур известных компаний, производящих графические процессоры, были выявлены ключевые особенности при построении графического ядра. Ведущие производители графических процессоров имеют общие концепции построения архитектур графических процессоров, но при этом каждая из архитектур имеет свои тонкости реализации.

Все графические процессоры основаны на большом количестве вычислительных блоков, способных вычислять огромный пласт данных, параллельно. Архитектура в построении работы вычислительных блоков основана на SIMD, потому что зачастую работа с данными подразумевает использование одних и тех же инструкций. Вместо использования больших и длинных команд, содержащих в себе набор инструкций, лучше использовать атомарные инструкции для более упрощенного взаимодействия с ядрами графического процессора.

AMD использует структуру ядра, где существуют универсальные арифметически-логические устройства (ALU), способные производить операции с плавающей точкой. AMD подразделяет 4 ALU в ядре на поток под работу с различными данными и 1 ALU под работу с данными, которые нужны всем потокам в ядре. Nvidia дробит ALU на два типа: для работы с целочисленными данными и для работы с вещественными данными, что сепарирует работу с адресацией и вычислением вещественных значений, требуемых для вычисления графики.

Ядра Nvidia обладают общим местом для хранения промежуточных и константных данных, называемый Регистровым файлом. Ядра AMD обладают локальной разделенной памятью, предоставляя всем потокам доступ к ней, в то время, как каждый поток ядра обладает собственными регистрами для сохранения константных и промежуточных значений. Оба производителя графических процессоров используют кешы для хранения данных, создавая некую иерархичность памяти. Целью использования кешей

вызвано ускорением работы с памятью и уменьшением простоя в прочтении данных с памяти, что крайне важно для достижения пиковой производительности видеокарты.

Обе компании стремятся к масштабируемости - последние архитектуры компаний не ограничиваются конкретным числом кластеров, все решения можно масштабировать, конфигурируя продукт под поставленные требования.

Все архитектуры являются конвейерными, чтобы обеспечить производительность и уменьшить простой отдельных элементов ядра.

На основе архитектур графических процессоров можно составить собственную первую итерацию графического ядра.

2. Архитектура ядра

В этом разделе описана разработка архитектуры графического ядра. Каждый отдельный компонент будет разобран до атомарной логики с подробным описанием механизма его работы. Постепенно описывая каждый элемент и связуя их между собой, в конце концов появится общая концепция архитектуры графического ядра. Разработка архитектуры ядра будет основана на обзоре существующих архитектур, а также знаний приобретенных из книг по разработке процессорных архитектур и цифровой схемотехники.

Некоторые наработки и принципы можно взять также из идей и наработок процессорных ядер, но для этого нужно различить структуру между архитектурами ядер CPU и GPU

2.1. Различие и сходство архитектур ядер CPU и GPU

Современные графические ускорители выполняют большое количество мелких задач одновременно. В отличие от центрального процессора (CPU), который предназначен для последовательного выполнения комплексных задач, графические ускорители спроектированы так, чтобы те выполняли множество небольших и независимых вычислений, таких как, к примеру отрисовки пикселей или обучение нейросетей. Одну такую мелкую задачу может выполнять специализированный блок, который называют ядром графического ускорителя. В современных видеокартах (GPU), количество ядер может скалироваться от нескольких тысяч до нескольких десятков тысяч.

Различие количества ядер у CPU и GPU является в том, что центральный процессор выполняет большой спектр различных, сложных и разнообразных задач, последовательно, в то время, как ядро GPU является узкоспециализированным ядром, который может выполнять лишь определенный список задач, которые приходят на обработку. Объем задач у CPU и GPU сильно отличается, ядро CPU обладает большим размером кеша, чем ядро GPU. Чем меньше логики у аппаратного блока и чем меньше для него создано памяти для хранения временных и локальных данных и инструкций, тем меньше размер аппаратного блока. Следовательно размер одного ядра CPU намного больше ядра GPU.

В отличие от ядер CPU, ядра GPU выполняют параллельно похожие операции над данными. В компьютерной графике, мультимедиа, криптографии и машинном обучении часто происходит ситуация, когда нужно работать с большим объемом данных,

которым требуется одна и та же обработка. Взять в пример вычисление какой-либо картинки на экране монитора - на сегодняшний день, разрешения экранов достигают более 2 миллионнов пикселей, что представляет собой уже огромный пласт работы, который необходимо обрабатывать чуть ли не за десятки миллисекунд. Поэтому, графические ускорители обладают маленькими, но многочисленными ядрами, тем самым позволяя работать с кучей данных, требующих одних и тех же инструкций за очень короткое время.

2.2. Интерфейсы графического ядра

Графическое ядро - сложный блок, который содержит в себе большое количество логики. Для успешного взаимодействия ядра необходимо определиться с входными и выходными портами блока. Одним из способов задачи входных и выходных портов блока является составление интерфейсов блока.

Интерфейс - это совокупность средств, правил и инструментов, которые обеспечивают взаимодействие между двумя системами [21]. В аппаратном мире существует огромное количество интерфейсов созданные под разный спектр задач. При разработке графического ядра в первую очередь нужно основополагаться на производительность ядра. Следовательно, чем быстрее будет происходить взаимодействие с памятью для ядра, тем меньше будет простоя во всем ядре. Это означает, что для реализации интерфейса для передачи данных необходимо выбирать такие интерфейсы, которые обеспечат минимальную задержку или же быструю передачу данных.

Разработка собственного интерфейса не является целесообразной, так как добавляет проблем, которые необходимо нужно будет решать на этапе построения основы интерфейса. Даже при создании собственного высокоскоростного интерфейса необходимо также потратить время на написание документации и верификации разработанного блока. К тому же, блоки с интерфейсами собственной реализации сложно портируемы в проекты, так как зачастую в реальных аппаратных проектах используются уже существующие интерфейсы различного типа. По все вышеперечисленным причинам выбор был сделан в использовании уже существующих интерфейсов

Интерфейсы AMBA (Advanced Microcontroller Bus Architecture или продвинутая архитектура шины микроконтроллера) - семейство интерфейсов, разработанные компанией ARM, предназначены для соединения различных блоков, процессоров и периферии внутри систем на кристалле (СнК) и микросхем ПЛИС.

Одним из таких интерфейсов является интерфейс AMBA AHB (Advanced High-performance Bus - продвинутая высокопроизводительная шина). Эта шина предназначена для высокоскоростных компонентов, таких как процессорные ядра, контроллеры памяти и прямой доступ к памяти (DMA). Данный интерфейс позволяет производить пакетную передачу, конвейеризацию операций и отдельные транзакции.

Также существует интерфейс под названием AMBA APB (Advanced Peripheral Bus - продвинутая периферийная шина) - предназначена для взаимодействия с низкоскоростной периферией, например, для взаимодействия с регистрами в периферии системы. Подобно АНВ, эта шина имеет фазы адреса и данных, но значительно урезанный несложный список сигналов. Интерфейс прост в разработке, но обладает недостаточной скоростью для передачи данных.

Последний среди трёх протоколов интерфейсов называется AMBA AXI (Advanced eXtensible Interface - продвинутый расширяемый интерфейс). Нацелен на высокопроизводительных и высокочастотных средств и включает возможности, которые делают интерфейс пригодным для высокоскоростных интерфейсов. Обладает отдельными фазами адреса/управления и данных, поддержкой передач невыровненных данных, применяя стробы байта, передачей на основе пакетов, с выдачей лишь начального адреса, выдачей множества внешних адресов с неупорядоченными ответами и легкостью добавления регистровых стадий для обеспечения близких задержек. Обладает очень сложной логикой и трудна в разработке [22].

2.2.1. Интерфейс передачи данных и инструкций (АНВ)

Графическое ядро, само по себе, не генерирует данные, с которыми оно будет работать. Это значит, что в ядро необходимо загрузить данные, с которыми она будет работать. Также немало важно уделить внимание инструкциям, ведь инструкции также должны поступать в графическое ядро, для того чтобы ядро знало, что необходимо сделать с пришедшими данными. Существуют различные интерфейсы для передачи данных, которые предоставляют возможность обмениваться данными между аппаратными блоками.

Для первой итерации графического ядра для передачи данных и инструкций будет достаточно интерфейса АНВ. В будущих итерациях стоит попробовать заменить интерфейс АНВ и AXI и сравнить их влияние на производительность всего ядра графического процессора.

2.2.2. Интерфейс передачи данных с регистровым файлом (АРВ)

Помимо передачи данных и инструкций в графическое ядро, ядро также должно выдавать какую-либо информацию о его состоянии. Один из способов - выводить все сигналы наружу. При таком подходе разработки портов графического ядра может возникнуть сложность с подключением: для сущностей, взаимодействующих с ядром (например обработчик прерываний), нет нужды отслеживать каждый такт состояния ядра. К тому же, в современных архитектурах графических процессоров количество ядер является очень большим и ядра образуют кластеры. Большое количество ядер с портами состояний усложняет подключение их всех к обработчикам ядер и отслеживанию состояний. Поэтому не стоит выводить всевозможные состояния ядра наружу - все данные можно хранить в специализированных регистрах. В данных регистрах можно сохранять статус ядра, включая его ошибки, состояние готовности. Помимо таких данных ядро может содержать в себе промежуточные значения для будущих вычислений. Данный набор структур называется регистровой картой и не только содержит в себе статусную информацию и промежуточные данные в ядре, но и управляющие сигналы. Как минимум, ядро должно каким-то образом запускаться, иметь возможность настроить собственный программный счётчик или отключить те или иные потоки в ядре. При грамотной разработке, в регистровой карте можно размечать отдельные регистры под разные спектры задач. Взаимодействие регистровой карты с внешними блоками должно также происходить через высокоскоростной протокол. Для общения с регистровой картой ядра достаточно разработать интерфейс АРВ для передачи данных и параметров ядра.

2.3. Поток

Графическое ядро должно содержать в себе вычислительные элементы, которые способны вычислять различные данные, приходящие в них. Элемент, способный вычислять полученные данные будет в дальнейшем называть потоком (thread). С помощью него, графика будет высчитываться. Количество потоков в ядре задаётся самим разработчиком. В различных архитектурах графических процессоров соотношение 1 поток к 1 ядру не применяется. Это вызвано тем, что часто возникает необходимость обрабатывать большой массив данных одной инструкцией. Большое количество потоков, исполняющие одну и ту же инструкцию, могут быстро выполнять вычисления над приходящими данными. Поэтому в одном ядре могут находиться большое количество

потоков, которые выполняют одну и ту же инструкцию над разными данными. Данное явление в разработке архитектуры называется SIMD.

2.3.1. Классификация параллельных вычислительных моделей (SIMT vs SIMD, MIMD).

Стоит понять концепцию и смысл выбора архитектуры параллелизма SIMD в архитектурах графических процессоров.

В разработке графических ускорителей принимают архитектурную классификацию параллелизма в потоках команд и данных SIMD, которая является одной из основных возможных архитектур параллелизма, основанных на таксономии Флинна. Среди основных классов наблюдаются следующие виды:

- ОКОД (SISD - single instruction, single data) - одиночный поток команд, одиночный поток данных;
- ОКМД (SIMD - single instruction, multiply data) - одиночный поток команд, множественный поток данных;
- МКОД (MISD - multiply instruction, single data) - множественный поток команд, одиночный поток данных;
- МКМД (MIMD - multiply instruction, multiply data) - множественный поток команд, множественный поток данных [23].

Все классы архитектур по Флинну представлены на рис. 15:

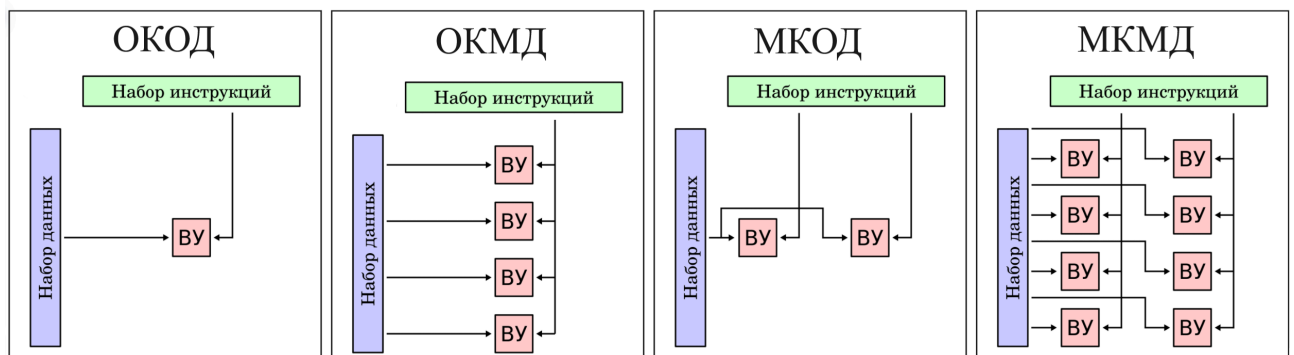


Рис. 15 — Все виды архитектур по таксономии Флинна

Примером архитектуры SISD является традиционный компьютер фон-Неймановской архитектуры с одним процессором, который выполняет последовательно одну инструкцию за другой, работая с одним потоком данных. SISD не является параллельной [24]. Основывать графический процессор на архитектуре SISD приведёт к слабой производительности, так как архитектура SISD вызывает ограничение скорости из-за своей природы. Первые процессоры основывались на этой архитектуре.

Архитектура SIMD позволяет обеспечить параллелизм на уровне данных, позволяя работать с большим количеством информации, над которой требуется работа лишь одной конкретной инструкции. [25] Технология SIMD не только улучшает производительность вычислительных систем, но и уменьшает площадь, ибо количество необходимых шин, для того чтобы связать аппаратные блоки существенно уменьшается. На основе этой архитектуры была разработана архитектура SIMT (Single Instruction Multiple Threads) - одна инструкция, множество потоков.

Архитектура MISD основана на том, что множество инструкций будет работать с одиночным потоком данных. Это может быть полезно в различных системах с отказоустойчивостью: в космонавтике или авионике. Множество процессоров работают над одним и тем же потоком данных. [26] В нейросетях и графике нет необходимости различными инструкциями выстраиваться в очередь для обработки одного потока данных. При такой архитектуре графическое ядро будет обладать большими задержками, что противоречит цели в создании производительной архитектуры графического ускорителя.

MIMD архитектура является распространенной архитектурой в компьютерах с несколькими процессорами. Благодаря такой архитектуре, можно обеспечить эффективное использование аппаратных ресурсов. При этом, ядра, которые будут использовать архитектуру MIMD будут сложными, т.к. на каждый поток команд и инструкций необходимо разработать блок выборки и декодирования команд, что увеличит место на кристалле, а также увеличит объем потребляемой энергии. Из-за такой архитектуры также необходимо разработать сложную систему планирования, потому что в MIMD могут возникнуть проблемы с взаимной блокировкой и состязанием за обладанием ресурсами. Это связано с тем, что потоки, пытаясь получить доступ к ресурсам, могут встретиться непредсказуемым способом на одном и том же ресурсе и такую проблему необходимо будет решать отдельными компонентами в архитектуре, которые также будут занимать место и энергию на кристалле [27].

Среди всех архитектур для разработки графического ядра явно подходит архитектура SIMD. На её основе будет разрабатываться архитектура графического ядра.

Заключение

Литература

- [1] И.А. Дудкин, Д.М. Старухин, и В.В. Черкасов, «Обзор и анализ 2D графических ускорителей».
- [2] И.А. Дудкин, Д.М. Старухин, и В.В. Черкасов, «Разработка архитектуры аппаратной реализации 2D графического ускорителя».
- [3] «Видеокарта». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/%D0%92%D0%B8%D0%B4%D0%B5%D0%BE%D0%BA%D0%B0%D1%80%D1%82%D0%B0>
- [4] «Nvidia». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/Nvidia>
- [5] «NVIDIA NV30». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/nvidia-nv30.g20>
- [6] Э. С. Ершов и Е. А. Сухотерин, «Архитектура графических процессоров Nvidia восьмого поколения с точки зрения вычислений общего назначения».
- [7] «NVIDIA Tesla A Unified Graphics and Computing Architecture». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-tesla-architecture.pdf>
- [8] «NVIDIA's Next Generation CUDA Compute Architecture». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-fermi-compute-architecture.pdf>
- [9] «Turing (микроархитектура)». [Онлайн]. Доступно на: [https://en.wikipedia.org/wiki/Turing_\(microarchitecture\)](https://en.wikipedia.org/wiki/Turing_(microarchitecture))
- [10] «NVIDIA TURING GPU ARCHITECTURE». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-turing-architecture.pdf>
- [11] «amd». [Онлайн]. Доступно на: <https://en.wikipedia.org/wiki/AMD>
- [12] «Part I The first fully programmable GPUs-a review». [Онлайн]. Доступно на: <https://www.jonpeddie.com/news/part-i-the-first-fully-programmable-gpus-a-review/>
- [13] «История развития графики AMD/ATI. Часть 2 путь универсальности». [Онлайн]. Доступно на: https://club.dns-shop.ru/blog/t-99-videokartyi/101511-istoriya-razvitiya-grafiki-amd-ati-chast-2-put-universalnosti/#sub_Radeon_HD2000_DirectX_10_i_superskalyarnaya_TeraScale
- [14] «R600-Family Instruction Set Architecture». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/ati-r600-isa.pdf>
- [15] «Evergreen Family Instruction Set Architecture Instructions and Microcode». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/ati-evergreen-isa.pdf>
- [16] «Введение в OpenCL». [Онлайн]. Доступно на: <https://habr.com/ru/articles/124925/>

- [17] «Учимся разрабатывать для GPU на примере операции GEMM». [Онлайн]. Доступно на: <https://habr.com/ru/companies/yadro/articles/934878/>
- [18] «AMD GrAPhICS CORES NEXT (GCN) ARCHITECTURE». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/amd-gcn1-architecture.pdf>
- [19] «Compute unit». [Онлайн]. Доступно на: <https://rocm.docs.amd.com/projects/omniperf/en/amd-staging/conceptual/compute-unit.html>
- [20] «RDNA (microarchitecture)». [Онлайн]. Доступно на: [https://en.wikipedia.org/wiki/RDNA_\(microarchitecture\)](https://en.wikipedia.org/wiki/RDNA_(microarchitecture))
- [21] «Интерфейс». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/%D0%98%D0%BD%D1%82%D0%B5%D1%80%D1%84%D0%B5%D0%B9%D1%81>
- [22] «АМБА». [Онлайн]. Доступно на: https://ru.wikipedia.org/wiki/Advanced_Microcontroller_Bus_Architecture
- [23] «Таксономия Флинна». [Онлайн]. Доступно на: https://en.wikipedia.org/wiki/Flynn%27s_taxonomy
- [24] «SISD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/SISD>
- [25] «SIMD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/SIMD>
- [26] «MISD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/MISD>
- [27] «MIMD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/MIMD>